OBJETIVOS DO TESTE?

Demonstrar que o teste está livre de defeitos? NÃO

Encontrar a existência de falhas!

Garantir conformidade com os requisitos

Os testes podem mostrar apenas a presença de defeitos, mas não a sua ausência.

Avaliar atributos como desempenho, usabilidade e segurança.

Sommerville:

- Demonstrar que o software atende aos requisitos.
- Descobrir situações em que o software falha ou se comporta incorretamente.

Pressman:

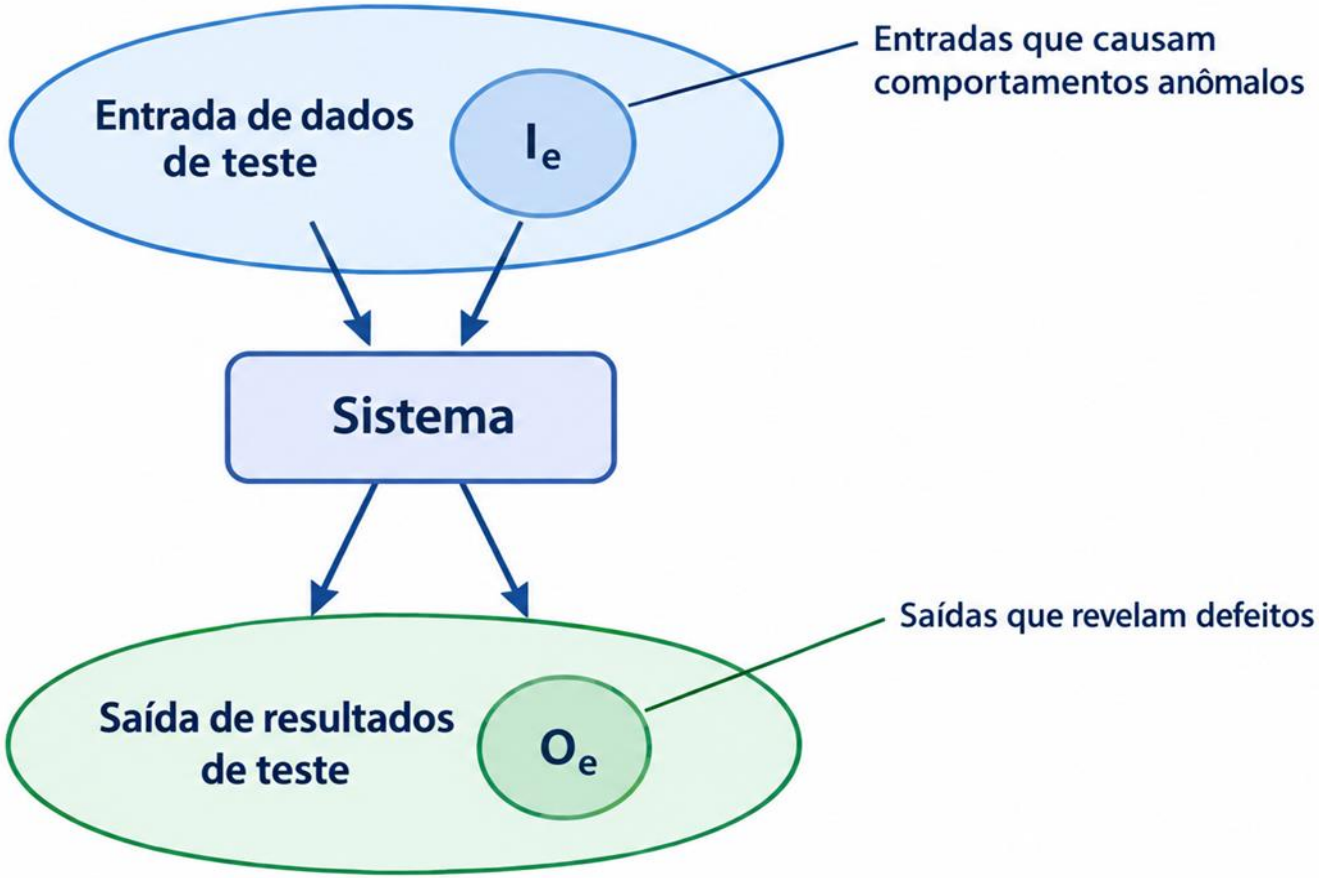
- Revelar erros lógicos e funcionais.
- Garantir a qualidade, confiabilidade e desempenho do software.

Boehm:

- Verificação: conformidade com especificações.
- Validação: atendimento às necessidades do usuário.

OBJETIVOS DO TESTE

Figura 8.1 Um modelo de entrada-saída de teste de programa



FASES E ESTÁGIOS DOS TESTES

Fase: relacionada ao momento do ciclo de vida do software em que o teste ocorre.

Estágio: etapa do processo de testes.

Enfoque: nível de decomposição ou integração dos componentes no momento do teste.

FASES DE TESTE

1)Teste de Desenvolvimento: durante a construção do software.

2)Teste de Release (ou Aceitação): antes da entrega para o cliente.

3)Teste de Usuário: após a entrega, no ambiente do cliente.

ESTÁGIOS DOS TESTES

1. Testes de desenvolvimento (unidade e componente)

2. Testes de sistema.

3. Teste de aceitação.

Figura 2.6 | Estágios de testes



ESTÁGIOS DOS TESTES

1. Testes de desenvolvimento. Ferramentas de automação de teste, como Junit.

2. Testes de sistema. Também visa mostrar que o sistema satisfaz seus requisitos funcionais e não funcionais.

ESTÁGIOS DOS TESTES

3. Testes de aceitação: validação final segundo os critérios do cliente.

NÍVEIS DE TESTE

Teste de Unidade

- Teste dos menores blocos de código (funções, métodos, classes).

Teste de Integração

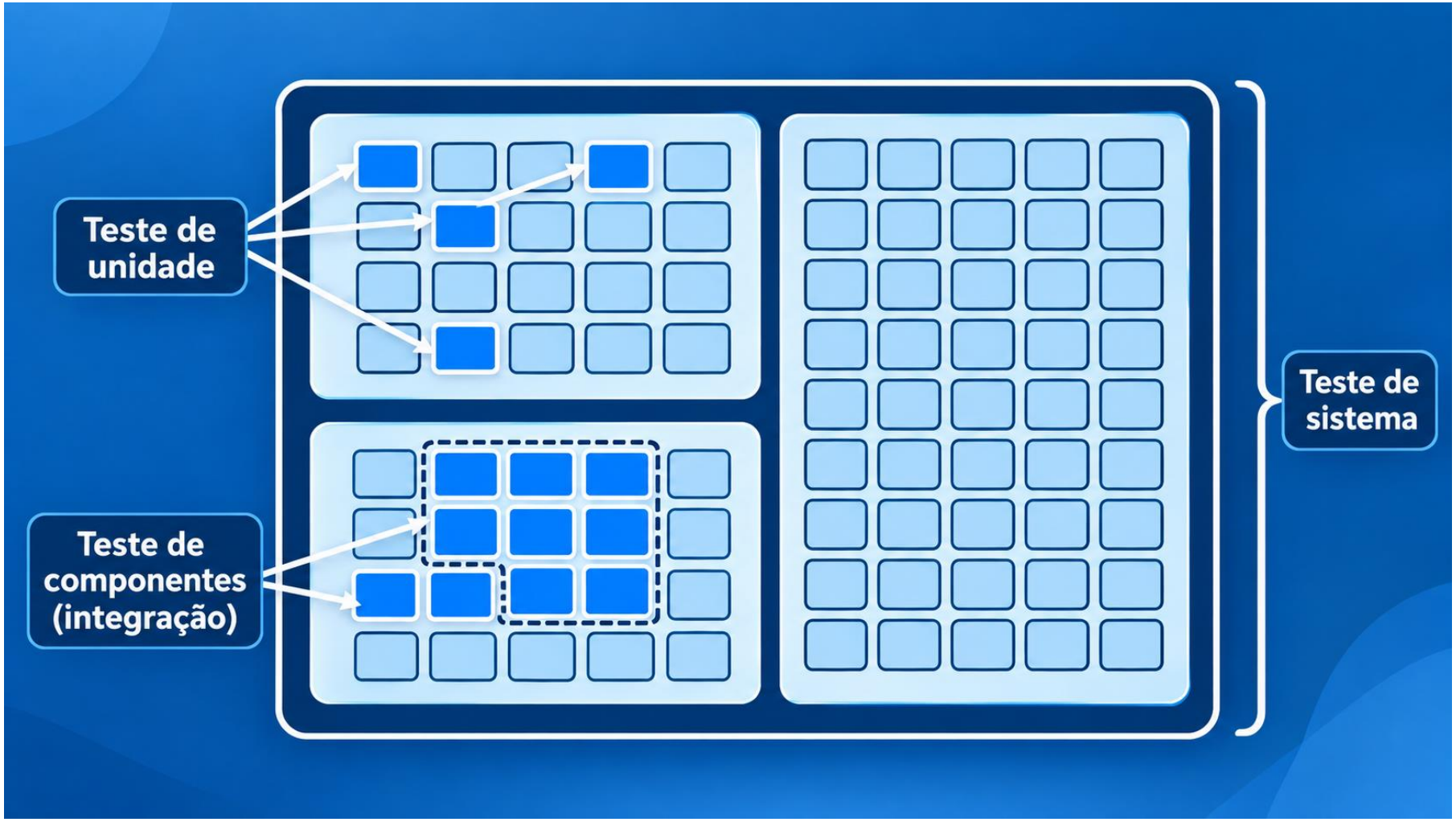
- Verifica a interação entre unidades de software.
- Identifica falhas na comunicação entre módulos.
- Pode ser:

  Incremental (top-down, bottom-up, sanduíche)

  Não incremental (big bang)

  Teste de Sistema
- **Avaliação do sistema como um todo.**
  - Inclui testes de desempenho, carga, estresse, segurança e usabilidade.

NÍVEIS DE TESTE



TÉCNICAS DE TESTE

Teste de caixa-preta (ou funcional):

- Avaliam a funcionalidade sem conhecer o código interno.
- Focam nas entradas e saídas.
- Técnicas:
  - Particionamento de equivalência
  - Análise de valor limite
  - Tabela de decisão
  - Teste de transição de estados
  - Teste baseado em casos de uso

Teste de caixa-branca: (ou teste estrutural)

TÉCNICAS DE TESTE

Teste de caixa-branca: (ou teste estrutural)

- Avaliam o funcionamento interno do software.
- Focam no fluxo de controle, estruturas de decisão e condições.
- Técnicas:
- Cobertura de comandos
- Cobertura de decisões/condições
- **Teste de caminhos independentes**

ESCOLHA DE CASOS DE TESTE UNITÁRIO

Os casos de teste devem mostrar que, quando usado como esperado, o componente que você está testando faz o que ele é proposto a fazer.

Se houver defeitos nos componentes, estes devem ser revelados por casos de teste.

Escolha de casos de teste unitário

Estratégias:

1. Teste de partição, se identifica os grupos de entradas que possuem características comuns e devem ser tratados da mesma maneira. Escolher os testes dentro de cada um desses grupos (números positivos, números negativos, etc.)

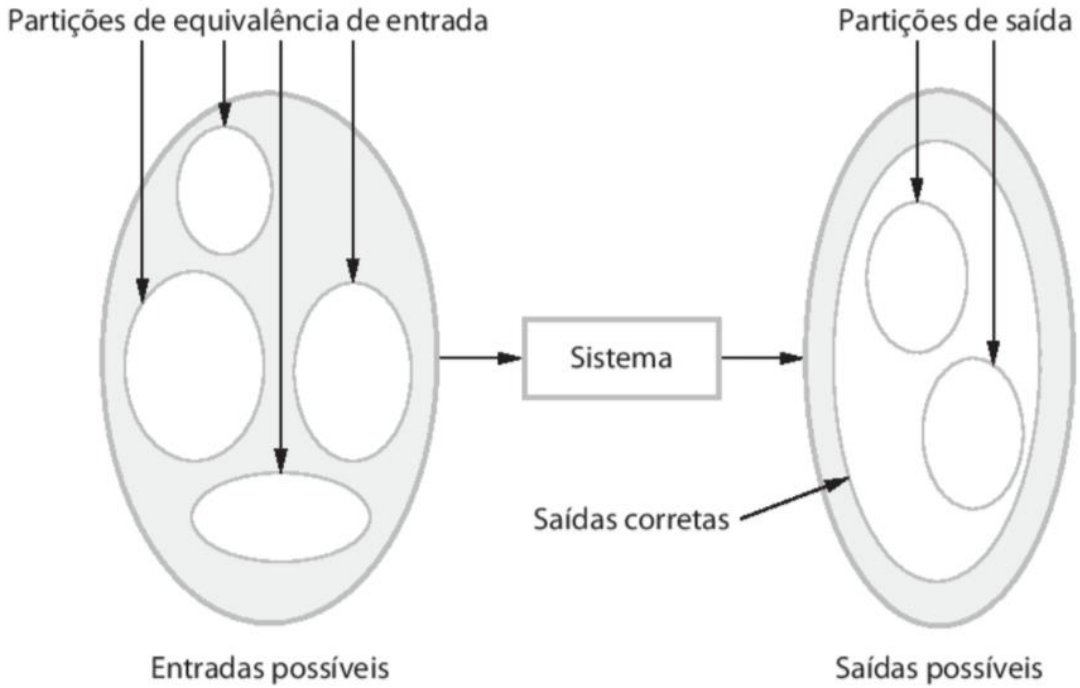
2. Testes baseados em diretrizes. Essas diretrizes refletem a experiência anterior dos tipos de erros que os programadores cometem frequentemente no desenvolvimento de componentes.

Particionamento de equivalência

Exemplo: programa que faz um cálculo com dois números positivos, se espera que o programa
se comporte da mesma forma para todos os números positivos.
Depois de ter identificado um **conjunto de partições**, se escolhe os casos de teste de cada uma.

Quando se usa a especificação de um sistema para identificar as partições de equivalência, isso é chamado **‘teste de caixa-preta’.** não precisa de nenhum conhecimento de como funciona o sistema.

Figura 8.5 Particionamento de equivalência



ESCOLHA DE CASOS DE TESTE UNITÁRIO

1. Usar Particionamento de Equivalência: Classe válida e Classe inválida

Classe de Equivalência	Exemplo	Saída
Idade < 13 (válida)	10	"criança"
13 ≤ Idade < 18 (válida)	15	"adolescente"
Idade ≥ 18 (válida)	20	"adulto"
Idade negativa (inválida)	-5	Deve levantar erro ou ser tratado
Idade não numérica (inválida)	"doze"	Deve levantar erro (TypeError)

DIRETRIZES DO TESTE DE PARTIÇÕES

- Testar o software com sequências de tamanhos extremos
  - Sequência de comprimento zero, único valor e tamanho máximo
- Usar sequências de tamanhos diferentes em testes diferentes
- Derivar testes para o primeiro, o médio e o último elementos da sequência

```
def verificar_par(numero):
    return numero % 2 == 0

import unittest

class TestVerificarPar(unittest.TestCase):
    def test_numero_par(self):
        self.assertTrue(verificar_par(2))
        self.assertTrue(verificar_par(4))

    def test_numero_impar(self):
        self.assertFalse(verificar_par(5))
        self.assertFalse(verificar_par(7))

    def test_numero_zero(self):
        self.assertTrue(verificar_par(0))

    def test_numero_negativo_par(self):
        self.assertTrue(verificar_par(-2))
        self.assertTrue(verificar_par(-6))
```

Classe de Equivalência

1) Números pares positivos
   2, 4
2) Números ímpares positivos: 5, 7
3) Zero (caso especial, mas considerado par): 0
4) Números pares negativos: -2, -6

RODANDO OS TESTES

No GBD

```
if __name__ == '__main__':
    suite = unittest.TestLoader().loadTestsFromTestCase(TestLimitesTriangulo)
    unittest.TextTestRunner(verbosity=2).run(suite)
```

Suite: coleção de testes automatizados reunidos para serem executados juntos

No Vscode, PyCharm

```
if __name__ == '__main__':    unittest.main()
```

EXERCÍCIO

O programa lê três valores inteiros que representam os lados de um triângulo. A partir dos valores, o programa informa se os lados formam um triângulo isósceles, escaleno ou equilátero.

Condição: a soma de 2 lados tem que se maior que o 3o lado

Conjunto de casos de teste adequado para testar este programa

EXEMPLO: AVALIAÇÃO

1. Existe casos de teste (c.t.) para triângulo escaleno válido?
2. Existe c.t. para triângulo isósceles válido?
3. Existe c.t. para triângulo equilátero válido?
4. Existem pelo menos 3 c.t. para isósceles válido contendo a permutação dos mesmos valores?
5. Existe c.t. com um valor zero?
6. Existe c.t. com um valor negativo?
7. Existe c.t. em que a soma de 2 lados é igual ao terceiro lado?
8. Para o item 7, tem um c.t. para cada permutação de valores?
9. Existe c.t. em que a soma de 2 lados é menor que o terceiro lado?
10. Para o item 9, tem um c.t. para cada permutação de valores?

EXEMPLO: AVALIAÇÃO

11. Existe c.t. para os 3 valores iguais a zero?
12. Existe c.t. com valores não inteiros?
13. Existe c.t. com número de valores errados, por exemplo, 2 valores ao invés de 3?
14. Para cada c.t. você especificou a saída esperada para a entrada projetada?

Questões baseadas em erros encontrados em implementações deste programa!!!

O programa deve receber três valores, a, b e c, que representam os comprimentos dos lados de um possível triângulo. Inicialmente, deve verificar se esses valores satisfazem a condição de existência de um triângulo: a soma de quaisquer dois lados deve ser maior que o terceiro lado. Assim, devem ser verdadeiras simultaneamente as condições a + b > c, a + c > b e b + c > a.
Se pelo menos uma dessas condições não for satisfeita, o programa deve informar que os valores não formam um triângulo. Caso as três condições sejam satisfeitas, o programa deve classificar o triângulo de acordo com a relação entre os comprimentos dos lados. Se os três lados forem iguais, o triângulo deve ser classificado como equilátero. Se exatamente dois lados forem iguais, deve ser classificado como isósceles. Se os três lados forem diferentes, deve ser classificado como escaleno.
Se chama: classificar_triangulo(a,b,c)

Criar casos de teste válidos e casos de teste inválidos: classes válidas e classes inválidas

ANÁLISE DE VALORES-LIMITE

Chamado Teste Fronteira ou *Boundary Value Analysis*

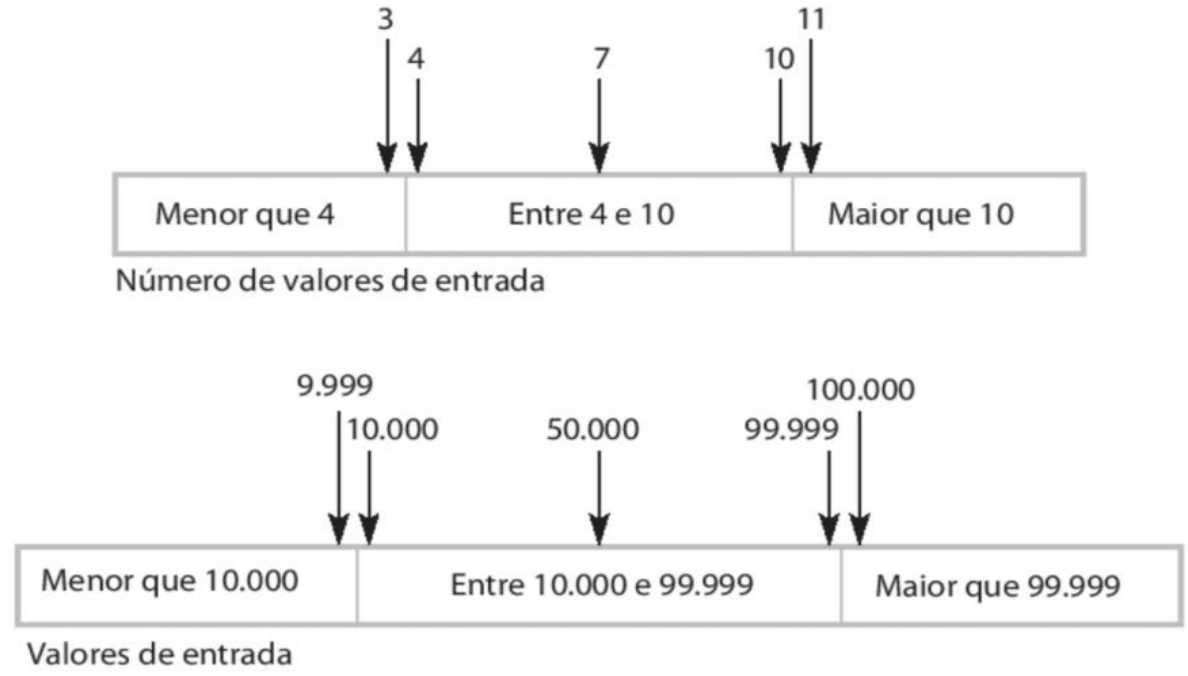
Critério de seleção que identifica valores nos limites das classes de equivalência:

- valor mínimo (máximo) igual ao mínimo (máximo) válido
- uma unidade abaixo do mínimo
- uma unidade acima do máximo
- arquivo vazio
- arquivo maior ou igual à capacidade máxima de armazenamento
- cálculo que pode levar a “overflow” (“underflow”)
- erro no primeiro (último) registro

PARTICIONAMENTO DE EQUIVALÊNCIA

Enquanto as partições dividem os dados em classes (ex: "Menor que 4", "Entre 4 e 10", "Maior que 10"), o **teste de fronteira** foca nos valores exatos que ficam nas linhas divisórias (limites) dessas classes.

Figura 8.6 Partições de equivalência



EXERCÍCIO: VALORES LIMITES

classificar_triangulo(a, b, c)

Casos válidos (formam triângulos):

Triângulo equilátero no limite mínimo:

- **Exemplo: a = 1, b = 1, c = 1**
- **Todos os lados são iguais e mínimos possíveis.**

Triângulo isósceles com dois lados iguais e próximo do limite:

- **Exemplo: a = 2, b = 2, c = 1**
- **Dois lados iguais, o terceiro no limite inferior.**

Triângulo escaleno com lados próximos ao limite:

- **Exemplo: a = 2, b = 3, c = 4**
- **Todos os lados diferentes e atendendo a desigualdade triangular.**

CASOS LIMITES- EXTREMOS

Se o programa aceita números muito grandes (exemplo: int do Python), testar limites superiores:

Triângulo equilátero com valores altos:

- **Exemplo: a = 10^6, b = 10^6, c = 10^6**

Triângulo isósceles com valores altos:

- **Exemplo: a = 10^6, b = 10^6, c = 10^5**

Teste de limites de precisão com valores quase iguais:

- **Exemplo: a = 1.000001, b = 1.000002, c = 1.000003**

Lado A	Lado B	Lado C	Resultado esperado
1	1	1	Triângulo equilátero
2	2	1	Triângulo isósceles
2	3	4	Triângulo escaleno
2	2	4	Não forma triângulo
1	2	4	Não forma triângulo
0	2	3	Não forma triângulo
-1	1	2	Não forma triângulo
10^6	10^6	10^6	Triângulo equilátero
10^6	10^6	10^5	Triângulo isósceles
1.000001	1.000002	1.000003	Triângulo escaleno (se suporte float)

```python
class TestLimitesTriangulo(unittest.TestCase):

    def test_equilatero_valores_altos(self):
        a = 10**6
        b = 10**6
        c = 10**6
        resultado = classificar_triangulo(a, b, c)
        self.assertEqual(resultado, "Equilatero")

    def test_isosceles_valores_altos(self):
        a = 10**6
        b = 10**6
        c = 10**5
        resultado = classificar_triangulo(a, b, c)
        self.assertEqual(resultado, "Isosceles")

    def test_limites_precisao_valores_quase_iguais(self):
        a = 1.000001
        b = 1.000002
        c = 1.000003
        #  Garante que o Python diferencia os valores flutuantes na precisão
        self.assertNotEqual(a, b)
        self.assertNotEqual(b, c)
        # C.T. Testa escaleno
        resultado = classificar_triangulo(a, b, c)
        self.assertEqual(resultado, "Escaleno")
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

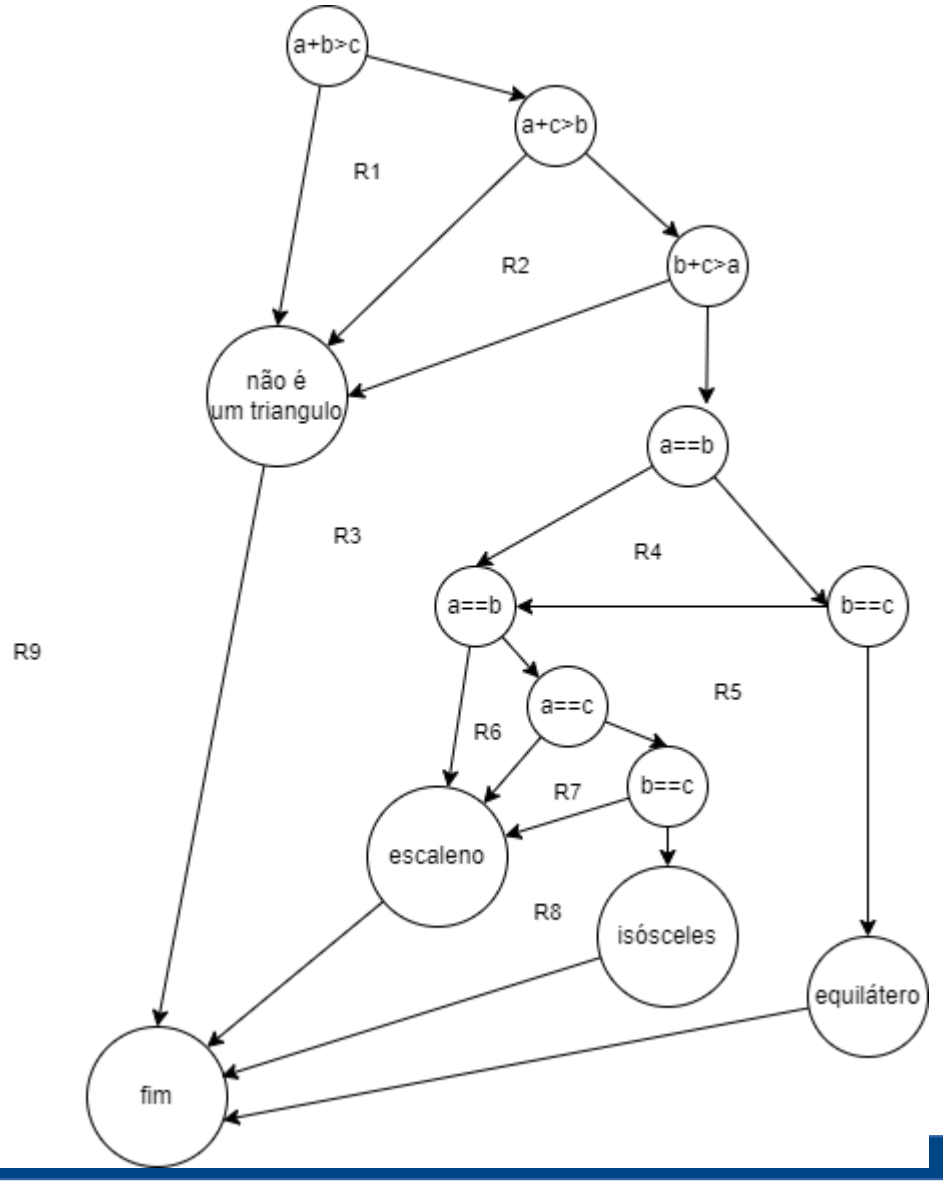
CASOS DE TESTE CAIXA PRETA

Lado A	Lado B	Lado C	Resultado esperado
1	1	1	Triângulo equilátero
2	2	1	Triângulo isósceles
2	3	4	Triângulo escaleno
2	2	4	Não forma triângulo
1	2	4	Não forma triângulo
0	2	3	Não forma triângulo
-1	1	2	Não forma triângulo
10^6	10^6	10^6	Triângulo equilátero
10^6	10^6	10^5	Triângulo isósceles
1.000001	1.000002	1.000003	Triângulo escaleno (se suporte float)

TESTE DOS CAMINHOS CAIXA BRANCA

```
def verifica_triangulo(a, b, c):

    if a + b > c and a + c > b and b + c > a:
        if a == b == c:
            return "Triângulo equilátero"
        elif a == b or a == c or b == c:
            return "Triângulo isósceles"
        else:
            return "Triângulo escaleno"
    else:
        return "Os valores não formam um triângulo"
```

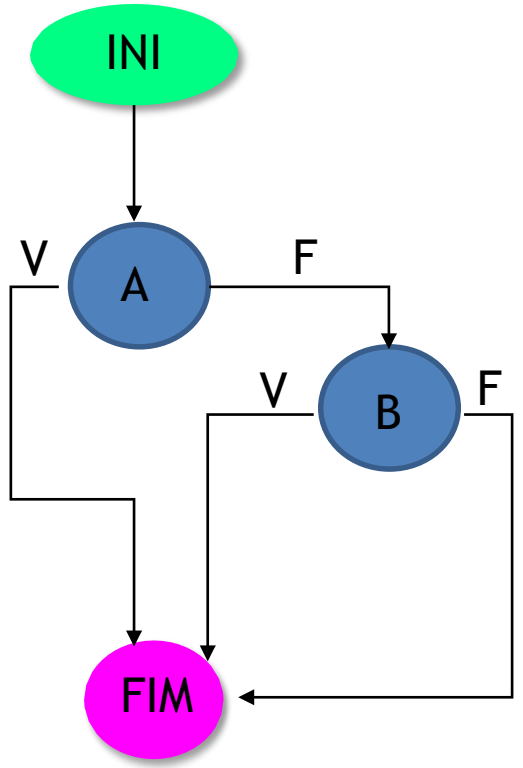


Revisão de Complexidade Ciclomática

Complexidade Ciclomática (Thomas McCabe, 1976)[1]

Métrica usada para medir o número de caminhos independentes em um código-fonte, indicando o quão complexo ele é para teste e manutenção.

```
def verifica_idade(idade):
    if idade < 18: #A
        return "Menor de idade"
    elif idade < 65: #B
        return "Adulto"
    else:
        return "Idoso"
```



Caminhos independentes:

INI-> A (V)-> Fim

INI -> A (F) -> B (V) -> Fim

INI -> A (F) -> B (F) -> Fim

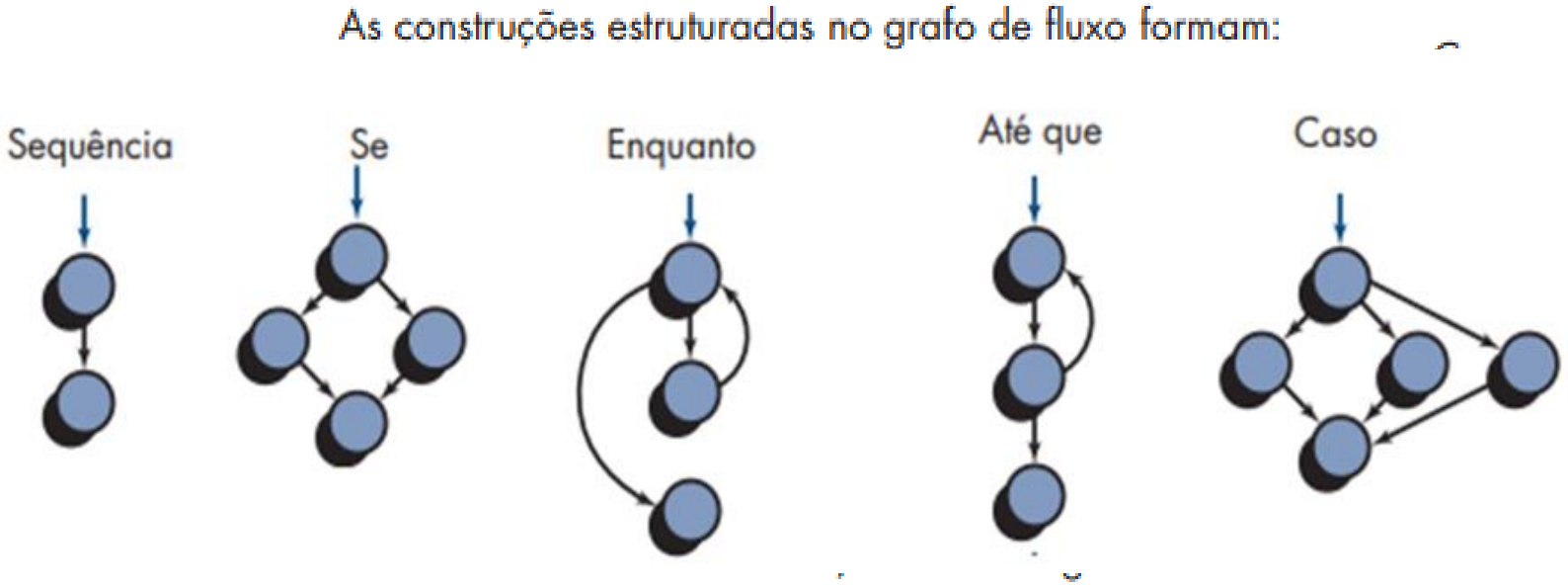
1. https://web.archive.org/web/20091229035408/http://classes.cecs.ucf.edu/eel6883/berrios/notes/Paper%204%20(Complexity%20Measure).pdf Acessado em 08/03/2025

Indicadores de Complexidade Ciclomática (Thomas McCabe, 1976)

Complexidade	Avaliação
1-10	Método simples. **Baixo risco**.
11-20	Método razoavelmente complexo. **Moderado risco**.
21-50	Método muito complexo. **Elevado risco**.
51-N	Método de **altíssimo risco** e bastante instável.

O que mostra a CC

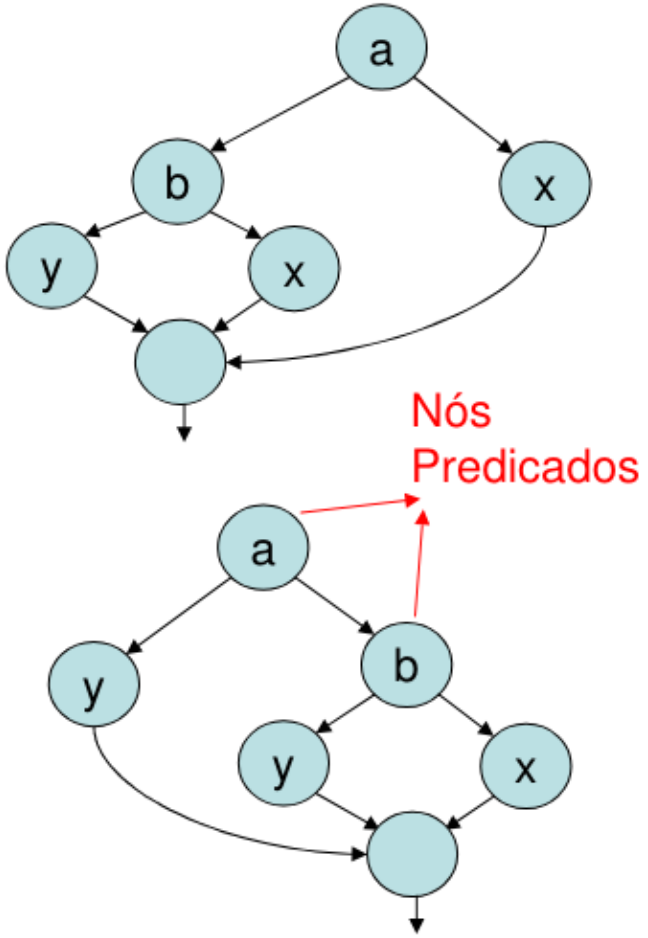
Notação simples para a representação do fluxo de controle, chamada de *grafo de fluxo* (ou *grafo de programa*).O grafo de fluxo representa o fluxo de controle lógico usando a notação:



Lógica Composta

```
SE a OU b
ENTÃO procedimento x
SENÃO procedimento y
FIM-SE
```

```
SE a E b
ENTÃO procedimento x
SENÃO procedimento y
FIM-SE
```



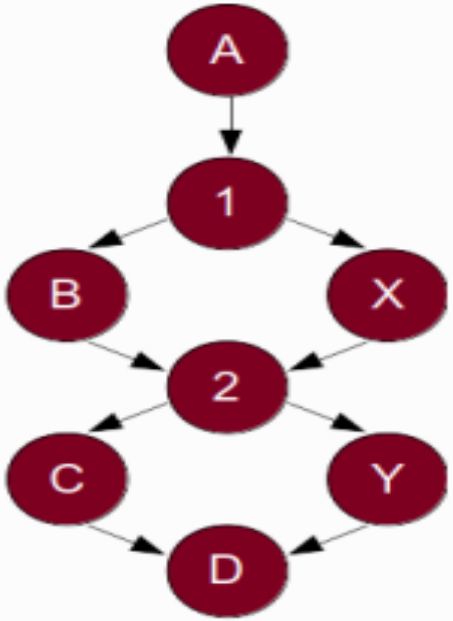
Grafo de Fluxo de Controle

Cada círculo (*nó),* representa um ou mais comandos. As setas no grafo de fluxo (*arestas),* representam fluxo de controle e são análogas às setas do fluxograma.

Fórmula V(G) = E – N + 2 onde E número de arestas e n número de nós.

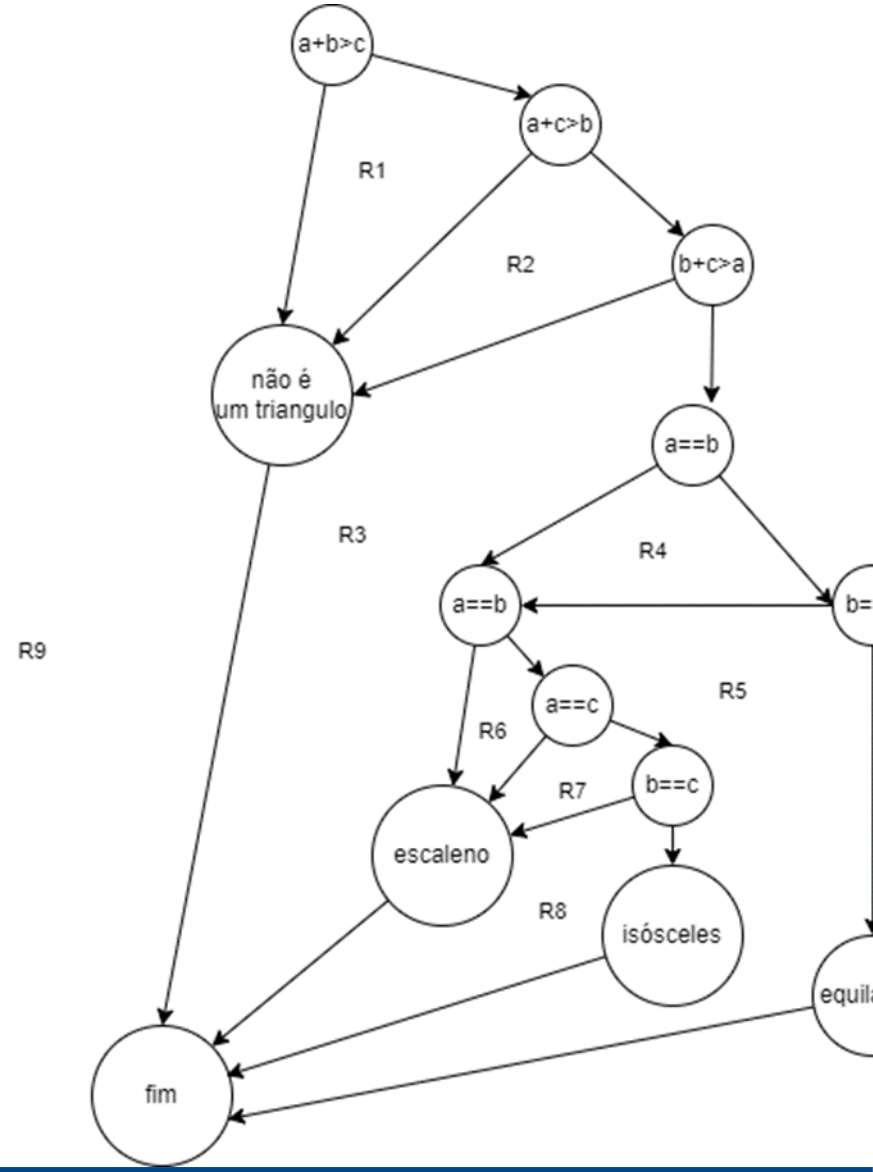
Ou V(G) = P + 1 nós predicados -> pontos de decisão no código (if, else if, while, for, switch-ca

```
print("A")
if (condition1)
  print("X")
else
  print ("B")
if (condition2)
  print("Y")
else
  print ("C")
print("D")
```



TESTE DOS CAMINHOS: CASOS DE TESTE

1. **Caminho 1:** Início → $a + b \leq c$ (falso triângulo) → Fim.
2. **Caminho 2:** Início → $a + b > c \rightarrow a + c \leq b$ (falso triângulo) → Fim.
3. **Caminho 3:** Início → $a + b > c \rightarrow a + c > b \rightarrow b + c \leq a$ (falso triângulo) → Fim.



TESTE DOS CAMINHOS: CASOS DE TESTE

Caminho 4: condição de triangulo válida e a==b==c, resultado: Equilátero → Fim.

Caminho 5: condição de triangulo válida a==b≠c ou a==c≠b ou b==c≠a, resultado: Isósceles → Fim.

Caminho 6: condição de triangulo válida. Todos os lados diferentes, resultado: Escaleno → Fim.

Caminho 7: condição de triangulo válida. Variação do caminho 4. a==b mas b b≠c e segue pelo caminho 5.

Caminho 8 condição de triangulo válida. Variação do caminho 5. a==b e a≠c, resultado: Escaleno → Fim.

Caminho 9: condição de triangulo válida. Variação do caminho 5. a==b e a==c e b ≠c, resultado Escaleno → Fim.